SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT-6

1. Student Details

StudentName	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A-2
Roll No.	202501110069
Date ofSubmission	24-10-2026

2. Practical Details

Practical No.	6
Practical Title	Implementation of Different Types of Inheritance in C++
Date Performed	
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

WriteaC++ program to implement the different types of inheritance: **single**, **multilevel**, **multiple**, **hierarchical** and **hybrid** inheritance. Use a **virtual base class** in the hybrid inheritance to avoid the ambiguity caused by the diamond problem.

4. Objectives

1. To understand the concept of inheritance and the relation between a base class and a derived class.
2. To implement single, multilevel, multiple, hierarchical and hybrid inheritance.
3. To understand the access of members in public, protected and private inheritance.
4. To understand the diamond problem and how a virtual base class solves it.
5. To understand code reusability using inheritance.

5. Expected Learning Outcomes

- Create a derivedclass from a base classusing the correct syntax.
- Identify and implement all five types of inheritance.
- Access the members of a base class through the object of a derived class.
- Use the keyword virtual to avoid duplicate copies of a base class in hybrid inheritance.
- Reuse existing code and reduce repetition in programs.

6. Theory / Concept

Inheritance: Inheritancei s a feature of object-oriented programming in which a new class (derived class or child class) acquires the properties and behaviour (data members and member functions) of an existing class (base class or parent class). It supports code reusability and represents an 'is-a' relationship. The syntax is class Derived : public Base { ... };

Access modes in inheritance: The mode used while inheriting (public, protected or private) decides how the members of the base class become accessible in the derived class. Private members of the base class are never directly accessible in the derived class.

Base class member	public inheritance	protected inheritance	private inheritance
public	public	protected	private
protected	protected	protected	private
private	not accessible	not accessible	not accessible

1. Single inheritance: One derived class inherits from only one base class. Example: Animal -> Dog.

2. Multilevel inheritance: A class is derived from a class which is itself derived from another class, forming a chain. Example: Grandfather -> Father -> Son.

3. Multiple inheritance: One derived class inherits from two or more base classes. Example: Writer and Singer -> Artist. The syntax is `class Artist : public Writer, public Singer.`

4. Hierarchical inheritance: Two or more derived classes inherit from a single base class. Example: Shape -> Rectangle and Triangle.

5. Hybrid inheritance: A combination of two or more types of inheritance. Example: Student -> Test and Sports -> Result is a combination of hierarchical and multiple inheritance.

Diamond problem and virtual base class: In hybrid inheritance, the class Result inherits from Test and Sports, and both of them inherit from Student. Without any special handling, Result receives two copies of the Student members, which creates ambiguity. When the base class is inherited using the keyword virtual (class Test : virtual public Student), only one shared copy of Student is passed to Result and the ambiguity is removed.

Order of constructors and destructors: Constructors are called from the base class to the derived class, and destructors are called in the reverse order (from the derived class to the base class). For a virtual base class, its constructor is called first.

7. Algorithm / Pseudocode

1. START
2. Single: create class Animal with eat() and class Dog derived from Animal with bark(); call both functions using a Dog object.
3. Multilevel: create classes Grandfather, Father (derived from Grandfather) and Son (derived from Father); call all three show functions using a Son object.
4. Multiple: create classes Writer and Singer and class Artist derived from both; call write() and sing() using an Artist object.
5. Hierarchical: create class Shape with setValues(); derive Rectangle and Triangle from it, each having its own area(); set values 10 and 5 and display both areas.
6. Hybrid: create class Student; derive Test and Sports from Student using the virtual keyword; derive Result from Test and Sports.
7. Read roll number, test marks and sports score from the user and store them in a Result object.
8. Display the roll number, test marks, sports score and total (marks + score).
9. STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Visual Studio Code (VS Code)
Libraries / Packages	iostream

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	
Repository Name	Assignment 6 C++
Branch Used	main
Commit ID / Latest Commit	
Dateof Last Commit	

10. Test Cases / Input and Output

Test Case	Input	ExpectedOutput	Actual Output	Status
1	101 80 15	Single, multilevel, multiple and hierarchical outputs are displayed (Rectangle area = 50, Triangle area = 25); hybrid: Roll No = 101, Test marks = 80, Sports score = 15, Total = 95	same as expected	pass
2	102 60 10	Single, multilevel, multiple and hierarchical outputs are displayed (Rectangle area = 50, Triangle area = 25); hybrid: Roll No = 102, Test marks = 60, Sports score = 10, Total = 70	same as expected	pass

```
PS C:\Users\Homesh> cd "c:\Users\Homesh\" ; if ($?) { g++ assi6.cpp -o assi6 } ; if ($?) { .\assi6 }
--- 1. Single Inheritance ---
Animal eats food
Dog barks

--- 2. Multilevel Inheritance ---
Grandfather owns the house
Father owns the shop
Son is a student

--- 3. Multiple Inheritance ---
Artist writes a song
Artist sings the song

--- 4. Hierarchical Inheritance ---
Area of Rectangle = 50
Area of Triangle = 25

--- 5. Hybrid Inheritance ---
Enter roll no, test marks and sports score: 101 80 15
Roll No      = 101
Test marks   = 80
Sports score = 15
Total        = 95
```

11. Results / Observations

TheC++ program wasimplementedin Visual Studio Code and all five types of inheritance worked correctly. In single inheritance, the Dog object could call both its own function bark() and the base class function eat(). In multilevel inheritance, the Son object could access the functions of Father and Grandfather. In multiple inheritance, the Artist object used the functions of both Writer and Singer. In hierarchical inheritance, Rectangle and Triangle shared the function setValues() of Shape and calculated their own areas. In hybrid inheritance, the virtual keyword gave only one copy of rollNo to the Result class, so there was no ambiguity and the total was calculated correctly. Both test cases gave the expected output.

12. Conclusion

The practicalsuccessfully demonstrated single, multilevel, multiple, hierarchical and hybrid inheritance in C++. It showed that inheritance allows a derived class to reuse the members of a base class, which reduces code repetition, and that a virtual base class is required in hybrid inheritance to avoid the diamond problem.

13. Assignment Questions

Q1.What is inheritance? Name and explain the types of inheritance in C++.

Answer: Inheritance is the process in which a derived class acquires the data members and member functions of a base class, so that existing code can be reused. The types are: single (one base and one derived class), multilevel (a chain of classes, A -> B -> C), multiple (one derived class with two or more base classes), hierarchical (many derived classes from one base class) and hybrid (a combination of two or more of these types).

Q2. What is the diamond problem? How does a virtual base class solve it?

Answer: The diamond problem occurs in hybrid inheritance when a class inherits from two classes that have the same base class. The last class then gets two copies of the base class members, and accessing them gives an ambiguity error. If the middle classes inherit the base class with the keyword virtual (class Test : virtual public Student), the last class receives only one shared copy of the base class and the ambiguity is removed.

Q3. Explain public, protected and private inheritance.

Answer: In public inheritance, public members of the base class stay public and protected members stay protected in the derived class. In protected inheritance, both public and protected members of the base class become protected. In private inheritance, both become private in the derived class. In all three modes, the private members of the base class are not directly accessible in the derived class. Public inheritance is the most commonly used mode.

Q4. What is the difference between multilevel and multiple inheritance?

Answer: In multilevel inheritance, classes are arranged in a chain, where each class is derived from only one class (Grandfather -> Father -> Son), so a class inherits indirectly from its grandparent. In multiple inheritance, a single class is derived directly from two or more base classes at the same level (Writer and Singer -> Artist). Multiple inheritance can cause ambiguity if both base classes have a member with the same name, whereas multilevel inheritance does not.

14. Student Declaration

Ihereby declarethatthe abovepractical work has been completed by me and that the implementation, analysis, and answers are my own work

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S